

FinAgent

An AI Agent Platform for Startup Evaluation and Financial Advisory

FinAgent applies large language model agents to two problems in early-stage finance: the cost of investor due diligence, and the gap between financial information and financial understanding. It runs a bounded research workflow that reads a pitch deck, researches the market, checks the company against the corporate registry, and produces a PDF report with charts — and an advisory assistant that routes a question to one of five specialist subagents. This report covers the need it addresses, how it works, how it is built, how to use it, and what it does and does not change.

Arav Saxena

September 2026

<https://github.com/arav7781/FinAgent>

Contents

1. Statement of Need	Why early-stage diligence is expensive, why a chatbot is not the fix, and what this project actually targets.
2. Technical Functionality	The two pipelines, node by node, and the design decisions inside each.
3. Architecture	Layering, module layout, the request path, and the deployment topology.
4. Usage and Scope	Installation, the two workflows in practice, the API surface, and what is deliberately out of scope.
5. Impact Overview	What changes for each user, the honest limitations, the risks, and where this goes next.
6. Appendix — Quick Reference	Technology stack, the complete API surface, configuration, and the commands needed to run everything.

Diagrams in this report are rendered from `docs/diagrams/generate_diagrams.py` and the document itself is rebuilt with `make report`, so no figure can drift from the code it describes.

1. Statement of Need

The problem in early-stage investment

Evaluating a seed-stage startup is labour that does not scale. An analyst reads the deck, sizes the market, identifies competitors, forms a view on the team, checks whether the company is legally registered and active, and writes the findings up. Done properly this is most of a working day per company. Done for a screening funnel of fifty applications, it is six weeks of one person's time before a single investment decision is made.

The organisations that most need this filter are the ones least able to staff it. A venture fund employs analysts. An angel syndicate, a university incubator, or an accelerator screening committee generally does not. The practical consequence is that the first filter becomes whatever is cheapest to apply — most often a warm introduction. That is a filter on a founder's network, not on the business, and it is a well-documented source of bias in who gets funded.

The gap on the other side

The same asymmetry appears in retail finance. Financial information is abundant and effectively free; financial understanding is not. A first-time investor searching for what an index fund is will find a hundred correct answers written for someone who already knows. Ask the same person how to allocate a portfolio and the honest answer depends on facts about them — risk tolerance, time horizon, goal — that no generic article can know.

These look like two problems. They share a structure: in both cases someone needs a judgement formed from scattered evidence, and the expertise to form it is expensive to buy. That is the shape of problem language model agents are genuinely suited to.

Why the obvious solution fails

The obvious approach — paste the deck into a chatbot and ask for an evaluation — fails in a specific and dangerous way. A language model asked to write a market analysis will write one whether or not it has evidence to write it from. Worse, thin evidence tends to produce the most confident prose, because there is nothing concrete to qualify. The result is a document with a total addressable market figure, a competitor list, and a recommendation, none of which are grounded in anything.

The core design problem. A fabricated report is worse than no report, because it looks like work. Someone downstream will treat a confident number as a researched one. Any system that automates diligence has to be built so that thin evidence produces a visibly thin report — not a confident one.

This reframes the engineering problem. The task is not to make a model write a good report; models already write fluent reports. The task is to build the surrounding structure that decides *whether the report should be written yet*, checks claims against sources that are not the model, and makes the limits of the analysis visible in the output.

Requirements

From the analysis above, the system must:

#	Requirement	Why it follows
R1	Ground analysis in supplied documents and live research, not model recall alone	Recall is stale and unattributable; a deck's numbers are the actual subject.
R2	Judge evidence sufficiency <i>before</i> writing, and act on the judgement	This is the direct countermeasure to the fabrication failure above.
R3	Verify factual claims against an independent authority	A model cannot tell whether a company legally exists. A registry can.
R4	Produce a durable, shareable artefact	An investment memo circulates. Chat scrollbar does not.
R5	Route advisory questions to behaviour appropriate to the question	Teaching, news reporting, and allocation advice have different correct behaviours — including refusing to answer yet.
R6	Refuse instruction-override and harmful input deterministically	A finance assistant is a target for prompt injection; model-based refusal can be argued with.
R7	Degrade to a working state when any external service is absent	A system that needs five live integrations to demonstrate anything cannot be evaluated or adopted.

The rest of this report traces each requirement to the mechanism that satisfies it.

2. Technical Functionality

FinAgent exposes two capabilities over one HTTP service: an evaluation pipeline that turns a startup profile into an investor PDF, and FinScope, an advisory router. Both are LangGraph state machines. Neither is a single prompt, and the reasons differ.

2.1 The evaluation workflow

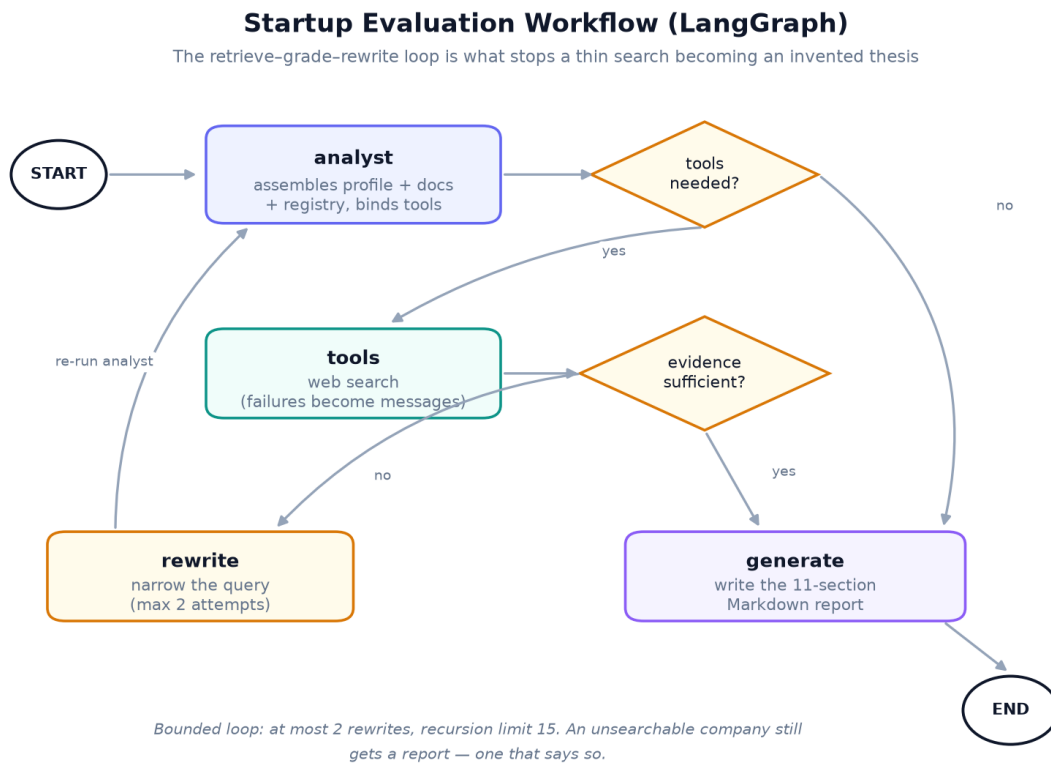


Figure 1. The evaluation workflow. The loop between retrieval and writing is the mechanism that satisfies requirement R2.

Execution begins at the **analyst** node, which assembles a prompt from three sources: the structured startup profile, document context retrieved for the query, and the company's registry verification status. It then binds a web search tool and lets the model decide whether it needs to use it. That decision is the model's, not a hard-coded step — a startup in a well-covered sector may need no external search at all.

If tools are called, the **tools** node runs them. It is wrapped rather than used directly, because an exception escaping this node would abort the entire evaluation. Instead, a failed search becomes a tool message that says so, which the model reads and works around. The distinction matters: the model learns that the search failed rather than silently receiving nothing.

The grader — the centre of the design

Tool output does not go straight to the report writer. It goes to **grade_relevance**, which asks the model one question: is this enough to write a comprehensive evaluation on? A 'no' routes to **rewrite**, which narrows the query and sends control back to the analyst for another attempt.

This loop is the direct answer to the fabrication problem in Section 1. Without it, a first-pass search returning nothing useful — the normal case for a pre-seed company with no press coverage — still produces a full report, and every specific number in it is invented. With it, the system tries again with a better query first.

Bounding the loop. A retry loop over a model that is judging its own inputs can run indefinitely. Three guards prevent it: a rewrite budget of two attempts held in graph state, a LangGraph recursion limit of 15, and a rule that a grader *failure* returns 'generate'. The last is deliberate — an infrastructure problem in the quality gate should not cost the user their report. A genuinely unsearchable company still gets one; it simply says the research was thin.

The **generate** node writes an eleven-section Markdown report: executive summary, problem and solution, market opportunity, competitive landscape, business model, team, traction, compliance, risks, investment thesis, and a recommendation. It prefers tool output as context; if every search failed it falls back to the analyst's own messages, so the document reflects the run that actually happened rather than an idealised one.

2.2 Document ingestion and retrieval

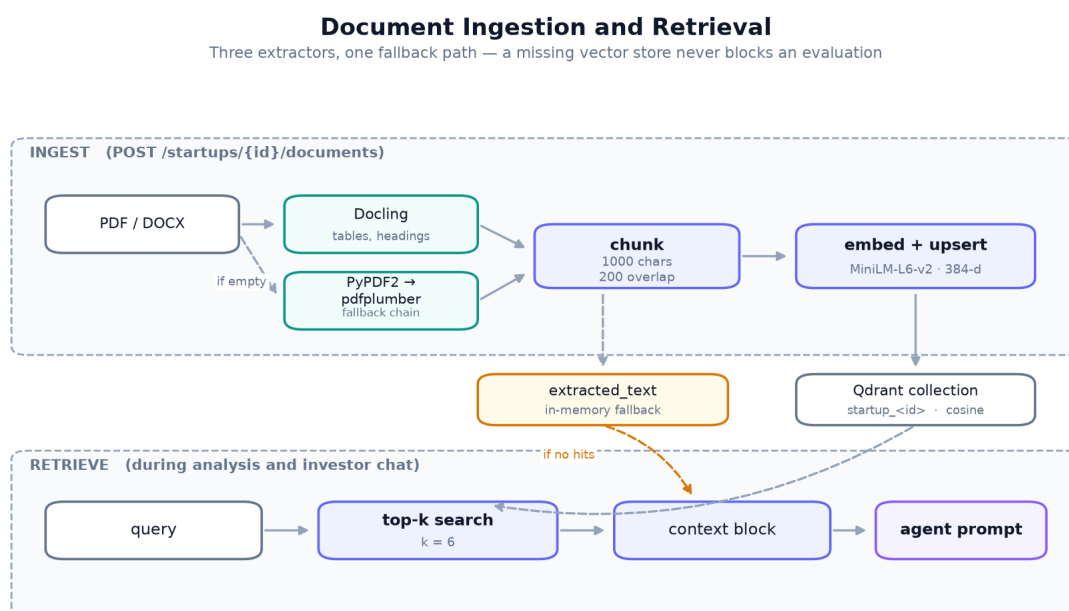


Figure 2. Ingestion and retrieval. Three extractors and two fallback paths, so a missing component degrades the result rather than blocking it (R7).

A pitch deck arrives as a clean digital PDF, a scanned export, or a DOCX, and no single parser handles all three well. Extraction therefore walks a chain: Docling first, because it preserves tables and headings as Markdown and a deck's numbers usually live in a table; then PyPDF2; then pdfplumber. The first extractor returning usable text wins and the rest are skipped.

Text is split into 1000-character chunks with 200 characters of overlap. The overlap is not arbitrary — without it, a claim spanning a boundary is retrieved as half a sentence, and half a financial claim is worse than none. Chunks are embedded with `all-MiniLM-L6-v2` (384 dimensions, running on CPU) and upserted into a per-startup Qdrant collection. Retrieval returns the six nearest chunks by cosine similarity.

Two fallbacks make this resilient. If the vector store or the encoder is unavailable at upload time, the raw extracted text is held on the startup record instead. If a retrieval returns no hits at analysis time, that stored text is used directly. The pipeline never fails because a vector database is down; it produces a less precisely grounded report and keeps going.

2.3 Registry verification

A model cannot determine whether a company legally exists. A registry can. FinAgent validates the 21-character Indian Company Identification Number locally, resolves it through a provider, and — the substantive step — compares the registry's answer with what the founder submitted.

That comparison is where the value is. It produces flags rather than a verdict:

Flag	What it means for an investor
MCA_NOT_VERIFIED	No successful verification is on record.
NAME_MISMATCH	The registered legal name differs from the name submitted. Often benign; occasionally not.
NO_DIRECTORS_FOUND	The registry lists no directors for this company.
COMPANY_STATUS_STRUCK_OFF	The company has been struck off the register.

Verification is advisory by design. A failed check annotates the report; it never blocks it. An unverified startup is still worth assessing, provided the assessment says it is unverified — which it does, on the cover of the PDF.

Running without a paid provider. Registry APIs are commercial. Without a key, FinAgent uses a deterministic sandbox in which a CIN whose serial ends in zero resolves to a struck-off company and everything else resolves to active. Both branches are therefore reachable and demonstrable offline. An earlier version keyed the failure case on a CIN ending in a letter, which the format validator rejects before the provider is ever called — the failure path was unreachable through the API. It is now covered by a regression test.

2.4 Report rendering

The PDF is assembled with ReportLab in three parts: a cover carrying the profile and the verification badge, a page of charts, and the model's analysis converted from Markdown to flowables. Three matplotlib figures are rendered head-less to PNG bytes and streamed directly into the document without touching disk — a TAM/SAM/SOM breakdown, an eight-year market trend, and a six-dimension scorecard covering product strength, market size, team quality, business model, competitive moat, and execution risk.

Stated honestly: the chart figures are illustrative baselines, not model-derived estimates. The dimensions and their weights are defined in `config/scoring_rubric.yaml`; connecting model-derived scores means replacing one mapping in `services/charts.py`. The PDF carries a disclaimer to this effect, which is the minimum a tool built to discourage fabrication owes its own output.

2.5 FinScope — the advisory router

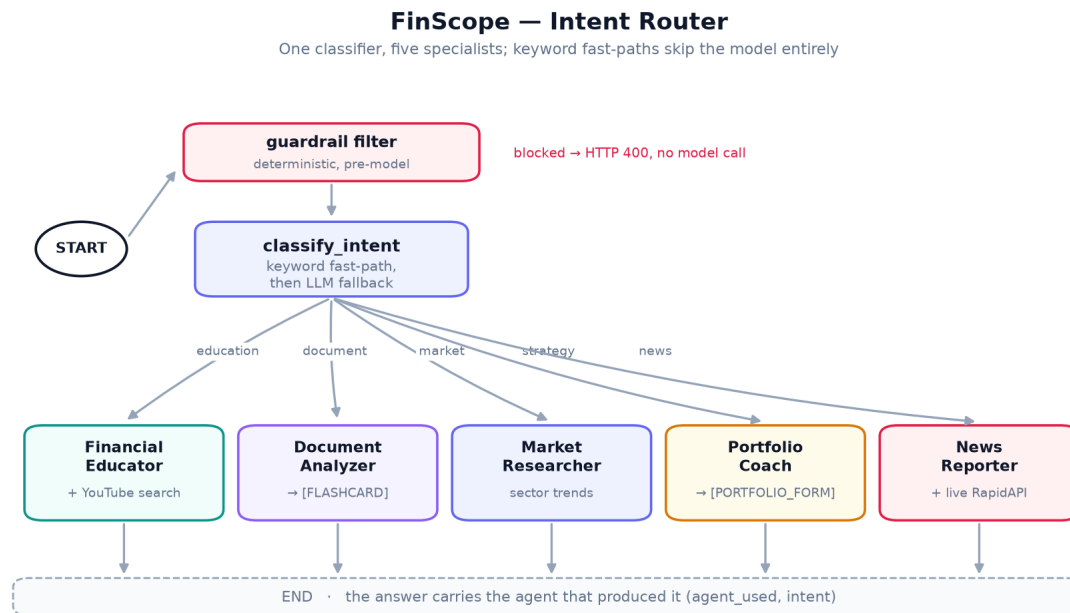


Figure 3. FinScope. A classifier and five specialists; keyword fast-paths resolve the obvious cases without a model call.

FinScope is a graph for a different reason. There is no loop — the structure is a fan-out, because the five things a retail investor asks for need genuinely different behaviour, and a single prompt trying to cover all five does each of them worse.

Subagent	Handles	Distinctive behaviour
Financial Educator	Concepts, terminology, tutorials	Runs a real YouTube search when videos are requested and weaves the results into the explanation.
Document Analyzer	Uploaded documents	Emits [FLASHCARD] and [COMPETITOR_CHART] blocks the frontend renders as widgets.
Market Researcher	Sectors, trends, market structure	Data-driven sector analysis.
Portfolio Coach	Allocation and strategy	Emits [PORTFOLIO_FORM] rather than advising without a risk profile.
News Reporter	Headlines and company updates	Fetches live articles <i>first</i> , then summarises only those.

Two decisions worth drawing out

The **News Reporter's ordering** is the point of that subagent. Live articles are retrieved before the model is invoked, and the prompt instructs it to use only the supplied data. A model asked for news without data will produce plausible headlines that do not exist — the same failure as an invented market size, applied to something a user is more likely to act on immediately.

The **Portfolio Coach's form** is a safety design as much as a user experience one. Allocation advice given without a risk tolerance, time horizon, or goal is advice that cannot be correct, because the inputs that determine the answer are missing. The agent asks rather than guesses, and only produces an

allocation table once it has been told.

Classification runs keyword fast-paths before the model. They are free, they are right about the obvious cases, and skipping a model call on 'latest Apple news' is the difference between a fast reply and a visible pause. The document fast-path deliberately requires a document to actually be loaded — otherwise 'tell me about this company' would route to an analyzer with nothing to analyse.

2.6 Guardrails

A deterministic pattern filter runs before any model call on the advisory endpoint. Blocked input returns HTTP 400 and never reaches an agent. It covers instruction overrides, prompt extraction, safety-bypass requests, jailbreak framings, regulated-professional personas, and harmful content.

It is regular expressions rather than a model, on purpose. It costs nothing, it cannot itself be talked out of a decision, and it fails closed. It is a first line of defence, not a complete safety system — the subagent prompts constrain behaviour further. The test fixtures pair prompts that must be blocked with benign questions that must not be, because a filter that refuses real questions is its own kind of failure.

3. Architecture

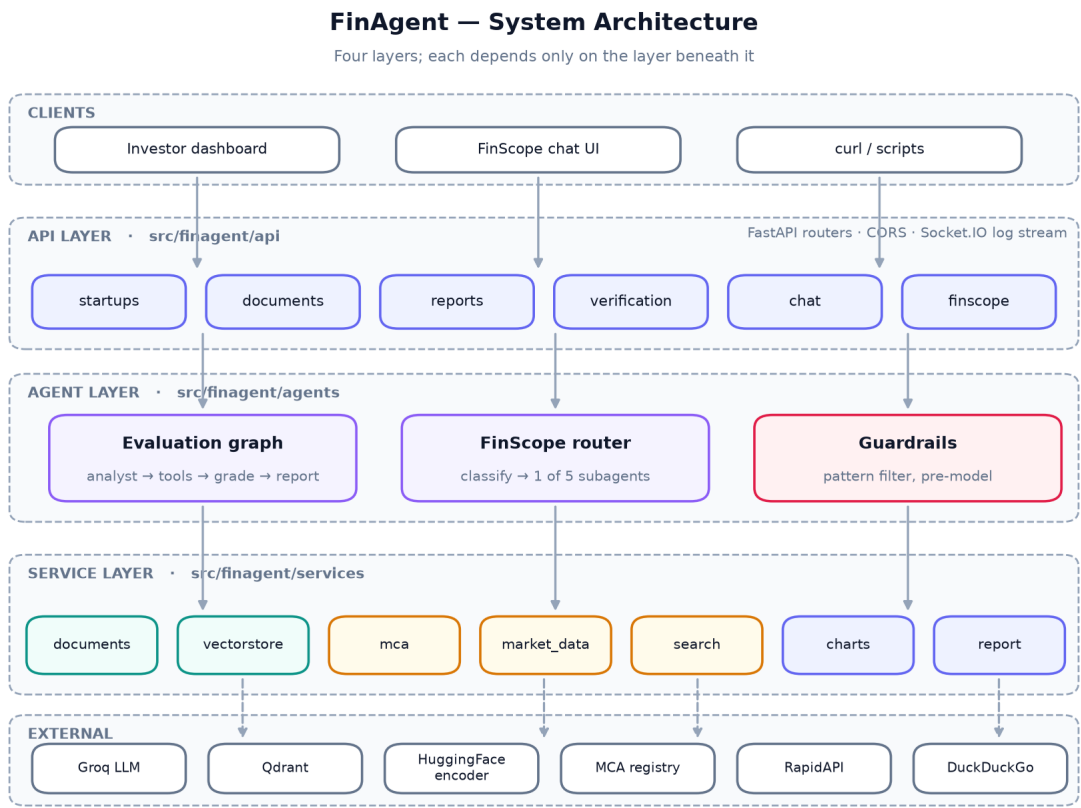


Figure 4. System architecture. Four layers, with dependencies pointing in one direction only.

3.1 Layering

FinAgent is a FastAPI application wrapped in a Socket.IO server, running two LangGraph workflows over a set of stateless services. It is organised in four layers, and the dependency rule is one-directional: a router never calls an external API directly, and a service never imports a router.

Layer	Package	Responsibility
API	<code>finagent.api</code>	HTTP concerns only — validation, status codes, serialisation.
Agents	<code>finagent.agents</code>	Graph topology, prompts, tools, guardrails.
Services	<code>finagent.services</code>	Documents, retrieval, registry, market data, charts, PDF rendering.
State	<code>finagent.store</code>	The in-memory repositories both layers read.

That constraint is what makes the service layer testable without a running server, and most of the test suite never starts one. Three modules sit outside the stack because everything uses them: `settings.py`, the only reader of the process environment; `schemas.py`, the wire contract; and `realtime.py`, the log stream.

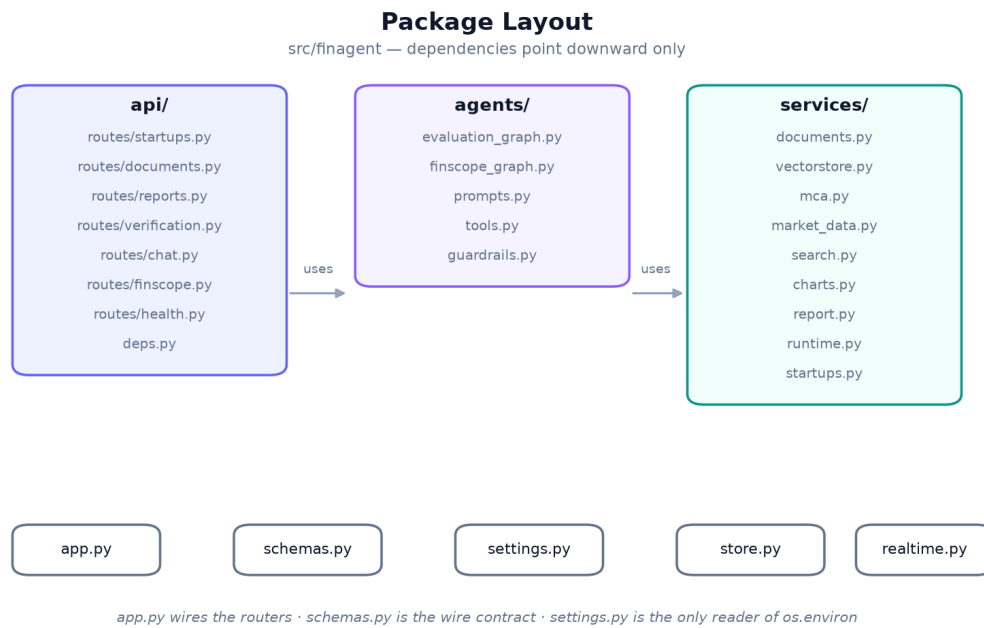


Figure 5. Package layout. Each router is one module; each capability is one service.

3.2 Request path

The diagram below traces a full evaluation. The two model calls are the latency: a complete run takes roughly 30 to 90 seconds depending on how many search rounds the grader demands.

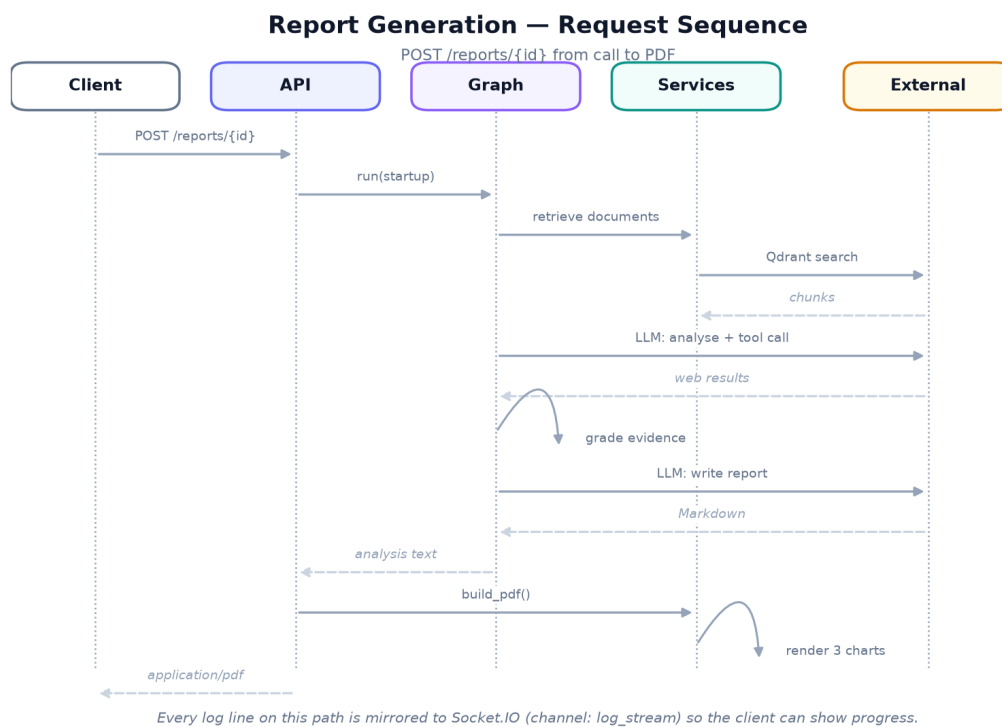


Figure 6. Report generation, end to end. Roughly 30–90 seconds, dominated by the two model calls.

Ninety seconds is longer than a browser request should stay open, so the same pipeline is also exposed asynchronously: a start endpoint returns immediately and a status endpoint is polled for structured sections. Throughout, every log record is mirrored to connected clients over Socket.IO. An agent pipeline is opaque from the outside, and streaming its progress is the difference between a user thinking the page is broken and a user watching the analyst search.

3.3 Configuration and degradation

Every environment variable is declared once, in `settings.py`, as a frozen dataclass. Feature code imports the settings object rather than reading the environment directly, which makes configuration discoverable in one place and testable by substitution.

Only the language model key is required. Every other integration is optional, and — per R7 — each failure path is specified rather than incidental:

Absent	Behaviour
Vector store	Document text is held on the startup record and passed to the agent directly.
Embedding model	Same fallback; semantic search is skipped.
Docling	Extraction falls through to PyPDF2, then pdfplumber.
Web search	The agent is told the search failed and writes from model knowledge — and the report says so.
Registry key	The deterministic sandbox provider answers.
Market data key	The News Reporter works from model knowledge; the headlines endpoint returns an empty list.

The health endpoint reports each dependency separately, so degraded operation is visible rather than silent. This is not defensive programming for its own sake — a system that requires five live third-party services before it will demonstrate anything is a system that cannot be evaluated, taught, or adopted.

3.4 Deployment

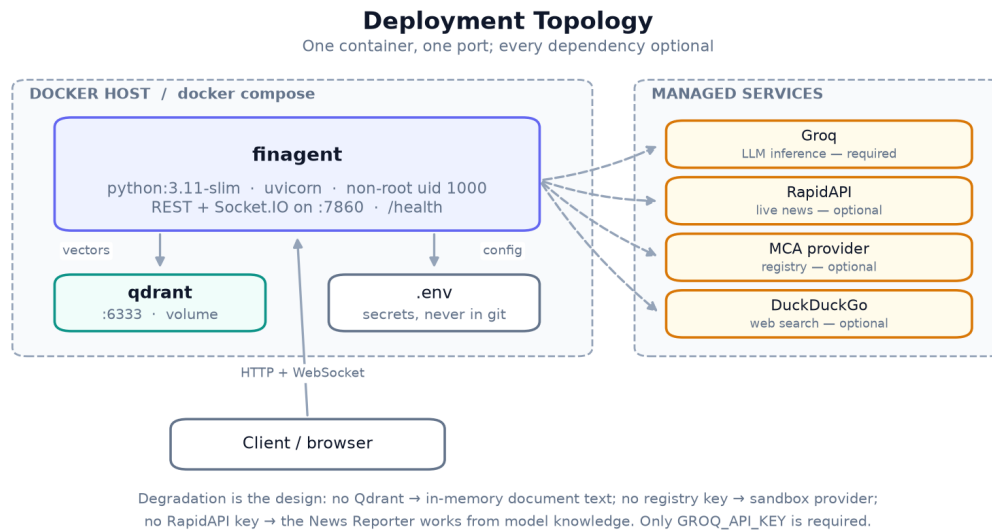


Figure 7. Deployment topology. One container, one port, every dependency optional.

The service ships as a single container based on `python:3.11-slim`, running as a non-root user, with dependencies installed in a layer above the source copy so application edits do not trigger a reinstall. A health check polls the health endpoint with a ninety-second start period, because the first boot downloads the embedding model. A Compose file adds a Qdrant instance with a persistent volume; removing it still leaves a working API.

3.5 State, and its limits

Startup records and analysis results live in process dictionaries behind an accessor module. This is a deliberate trade for an evaluation build: it removes database provisioning from the critical path of getting the system running, and the accessor functions are the single seam where a real database would go. Nothing outside that module touches the dictionaries.

The consequence, stated plainly: state does not survive a restart, and the service does not scale beyond one process. For the intended use — evaluation, demonstration, and single-analyst workflows — this is acceptable. For anything else it is the first thing that must change, and the codebase is arranged so that it is a contained change.

4. Usage and Scope

4.1 Installation

```
git clone https://github.com/arav7781/FinAgent.git
cd FinAgent

cp .env.example .env      # add GROQ_API_KEY
make install              # pip install -r requirements.txt
make run                  # http://localhost:7860
```

Interactive API documentation is served at `/docs`. With Docker, `docker compose up --build` brings up the service together with a Qdrant instance. Local installation needs a few system libraries for document conversion and chart rendering; these are listed in the deployment documentation.

4.2 The evaluation workflow in practice

Four calls take a startup from registration to a finished report. `examples/01_evaluate_startup.py` performs all four.

Step	Call	Result
1	POST <code>/startups</code>	Registers the profile and returns an id.
2	POST <code>/api/startups/{id}/documents/upload</code>	Extracts, chunks, embeds and indexes the pitch deck.
3	POST <code>/startups/{id}/verify/mca</code>	Resolves the CIN and records consistency flags.
4	POST <code>/reports/{id}</code>	Runs the pipeline and streams back the PDF.

Step 4 has an asynchronous counterpart for user interfaces that cannot block: `POST /api/startups/{id}/analyze` starts the run and `GET /api/startups/{id}/report/status` is polled for parsed sections. Once a startup is registered, `POST /chat/{id}` answers investor questions grounded in its profile and documents.

4.3 The advisory workflow in practice

```
curl -X POST http://localhost:7860/finscope/chat \
  -H 'Content-Type: application/json' \
  -d '{"question": "What is an index fund?", "session_id": "demo"}'

{"answer": "...", "agent_used": "Financial Educator", "intent": "education"}
```

The response names the subagent that handled the message, so a user interface can show which specialist answered. Uploading a document through `POST /finscope/analyze-document` caches its text against the session, and follow-up questions in the same conversation resolve without a re-upload.

4.4 Repository organisation

Path	Contents
<code>src/finagent/</code>	The application package — api, agents, services, schemas, settings, store.
<code>tests/</code>	125 tests; no network, no API key, no database required.

Path	Contents
examples/	Four runnable walkthroughs plus a file of curl recipes.
dataset/	Synthetic startups, CIN cases, 30 labelled intents, 20 guardrail probes.
docs/	Architecture, API reference, agents, configuration, deployment, testing, and the diagram generator.
config/	Logging configuration, tunable defaults, and the scoring rubric.
report/	This document.

4.5 Verification and testing

The suite covers the HTTP contract, CIN validation and every consistency flag, intent routing, the rewrite budget and grader-failure handling, chunking and extractor fallback order, chart and PDF generation, repository semantics, symbol resolution and cache behaviour, and configuration defaults.

It runs offline. A placeholder key is set before any import so settings build, and the model is stubbed wherever a test exercises a model path. The API tests deliberately run against the application with no dependencies initialised, which proves exactly which endpoints work when everything optional is down. Live third-party calls are not tested — they are slow, non-deterministic and metered — so the seams around them are tested instead, and the example scripts serve as the integration check.

```
$ make test
125 passed in 2.08s

$ make lint
All checks passed!
```

4.6 Scope

In scope

- Automated first-pass evaluation of early-stage startups from a profile and supporting documents.
- Corporate registry verification with claim-versus-record consistency checking.
- Investor question answering grounded in a specific company's materials.
- Financial education, document explanation, market commentary, portfolio guidance, and live news summarisation.
- A durable PDF artefact suitable for circulation within a screening committee.

Out of scope, deliberately

- Investment decisions.** The output is a first-pass filter and a structured brief. A recommendation in the report is an input to human judgement, not a substitute for it.
- Regulated financial advice.** FinScope explains concepts and structures questions. It is not a registered adviser and does not recommend specific securities.
- Real-time trading or execution.** There is no brokerage integration and none is planned.
- Multi-tenant production operation.** There is no authentication, CORS is open, state is in process memory, and there is no rate limiting. These are enumerated in the deployment documentation as prerequisites for public exposure.
- Model training.** The system orchestrates a hosted model and a sentence encoder. Nothing here is trained or fine-tuned.

5. Impact Overview

5.1 What changes, and for whom

User	Before	With FinAgent
Angel investor / syndicate	Reads decks personally; the funnel is capped by available hours.	A structured brief per company with registry status attached, so the reading time goes to shortlisted companies.
Accelerator screening committee	Inconsistent evaluation across reviewers and cohorts.	The same eleven sections and the same six dimensions for every applicant, which makes comparison meaningful.
Founder	Diligence feedback arrives late, if at all.	The same analysis can be run on one's own deck before pitching, surfacing the questions an investor will ask.
Retail investor	Generic articles written for people who already understand them.	Explanation at the asked-for level, with the portfolio agent refusing to advise until it knows the risk profile.

5.2 The measurable effect

The concrete claim is narrow and worth stating precisely. A first-pass evaluation that occupies an analyst for the better part of a day — reading, market sizing, competitor identification, registry lookup, write-up — is produced in roughly a minute and a half of machine time, in a consistent format, with the registry check performed rather than skipped.

What that does *not* claim is that the output equals the analyst's. It does not. It changes what the analyst's day is spent on: instead of constructing a first draft for fifty companies, the time goes to interrogating the eight that survive the filter. The value is in reallocation, not replacement.

The second-order effect

If a structured first pass is cheap enough to run on everything, the screening filter can be the business rather than the network. That is a small contribution to a documented problem in venture funding — that access to capital correlates with access to introductions — and it is the part of this project that would matter most if it were adopted at any scale.

5.3 Limitations

Stated directly, because a system built to discourage overconfident output should not produce an overconfident report about itself.

Limitation	Consequence and mitigation
The written analysis is model-generated and can be wrong.	Mitigated by evidence grading, registry verification, and an explicit disclaimer in the PDF — reduced, not eliminated.
Chart figures are illustrative baselines, not derived estimates.	Stated in the report and in the repository. The rubric is externalised in configuration so real scoring is a contained change.

Limitation	Consequence and mitigation
State does not survive a restart.	Acceptable for evaluation use; the store module is the single seam where a database goes.
Registry verification is India-specific (CIN).	The provider interface is isolated in one service module; other jurisdictions are an implementation of the same shape.
Web search depends on an unofficial free endpoint.	Three transports are tried; if all fail the agent is told, and the report says the research was thin rather than hiding it.
No authentication, open CORS, no rate limiting.	Enumerated in the deployment documentation as prerequisites before any public exposure.

5.4 Ethical considerations

Three risks are worth naming explicitly.

Automation bias. A well-formatted PDF with charts reads as more authoritative than the evidence behind it. Every countermeasure in the design — the grader, the verification badge, the disclaimer, the report explicitly noting thin research — exists to work against this, and none of them fully solves it. Presentation quality and evidence quality are independent, and a reader has to be told so.

Inherited bias. A model evaluating a team is drawing on training data that encodes which founder profiles have historically been funded. Applying it uniformly at least makes the criteria consistent and inspectable, which is more than an informal network filter offers — but consistency is not fairness, and this system should not be the final word on a funding decision.

Advice boundaries. The Portfolio Coach withholds allocation advice until it knows the user's risk tolerance, horizon and goal, and the guardrail filter blocks attempts to have the assistant role-play a regulated professional. Neither makes this a registered adviser, and the scope section says so.

5.5 Further work

- Model-derived scorecard.** Have the generation node emit structured scores against the rubric in `config/scoring_rubric.yaml` so the chart reflects the analysis rather than a baseline.
- Persistence.** Replace the in-memory store with PostgreSQL behind the existing accessor interface; the seam is already in place.
- Citation tracking.** Attach source attribution to each retrieved chunk so the report can cite which part of the deck a claim came from.
- Broader registry coverage.** Companies House and equivalents, behind the same provider interface.
- Evaluation harness.** Score generated reports against analyst-written ones on a labelled set, to replace assertion with measurement.
- Authentication and multi-tenancy,** as the precondition for any deployment beyond a single user.

5.6 Conclusion

The central argument of this project is that applying a language model to financial analysis is mostly an exercise in building the structure around the model, not in prompting it. Models write fluent investment memos on no evidence at all; the engineering that matters decides whether there is enough evidence to write one, checks the claims that can be checked against something other than the model, and makes the limits of the analysis visible in the output rather than hiding them behind good typography.

FinAgent implements that argument in a specific, working form: a bounded retrieve–grade–rewrite loop, independent registry verification with claim-versus-record flagging, live data fetched before the model is asked, deterministic input filtering ahead of any model call, and a specified degradation path for every external dependency. The result is a system that produces a genuinely useful first-pass evaluation, and that is honest about the difference between a first pass and a decision.

6. Appendix — Quick Reference

A. Technology stack

Concern	Choice	Why
Web framework	FastAPI + Uvicorn	Async, and generates the OpenAPI schema from the same Pydantic models used for validation.
Agent orchestration	LangGraph	Explicit state machines. The evaluation loop is a graph edge, not control flow buried in a function.
Language model	Groq — Llama 4 Scout 17B	Fast inference with tool calling, which the analyst node requires.
Embeddings	all-MiniLM-L6-v2 (384-d)	Runs on CPU, so retrieval needs no GPU.
Vector store	Qdrant	One collection per startup keeps tenants naturally isolated.
Document conversion	Docling, PyPDF2, pdfplumber	Three parsers because no single one handles clean, scanned and DOCX decks well.
PDF rendering	ReportLab + matplotlib	Charts render head-less to bytes and stream straight into the document.
Realtime	python-socketio	Mirrors server logs to clients so a long agent run is visible.
Testing	pytest + httpx	125 tests, offline, no keys required.

B. API surface

Endpoint	Method	Purpose
/startups	POST / GET	Register a startup · list all
/startups/{id}	GET / PUT	Read · update a profile
/startups/{id}/documents	POST	Ingest a document from a URL
/api/startups/{id}/documents/upload	POST	Ingest an uploaded file
/startups/{id}/verify/mca	POST / GET	Verify by CIN · read stored status
/reports/{id}	POST	Run the pipeline, return the PDF
/api/startups/{id}/analyze	POST	Start a background analysis
/api/startups/{id}/report/status	GET	Poll analysis status
/chat/{id}	POST	Investor Q&A by startup id
/chat/by-name/{name}	POST	Investor Q&A by startup name
/finscope/chat	POST	Advisory chat through the intent router
/finscope/analyze-document	POST	Document to flashcard analysis
/api/finance-news/headlines	GET	Raw live headlines for a symbol

Endpoint	Method	Purpose
/health	GET	Dependency readiness

C. Environment variables

Variable	Required	Effect when absent
GROQ_API_KEY	Yes	No agent can run.
QDRANT_URL	No	Retrieval falls back to in-memory document text.
QDRANT_API_KEY	No	Only needed for Qdrant Cloud.
RAPIDAPI_KEY	No	News Reporter uses model knowledge; headlines endpoint returns [].
MCA_API_KEY	No	The deterministic sandbox registry answers.
MCA_API_URL	No	Defaults to the Karza company-master endpoint.
LLM_MODEL	No	Defaults to Llama 4 Scout 17B.
EMBEDDING_MODEL	No	Defaults to all-MiniLM-L6-v2.
PORT / HOST / LOG_LEVEL	No	Default to 7860 / 0.0.0.0 / INFO.

D. Commands

Command	Does
make install	Install runtime dependencies
make dev	Install runtime and development dependencies
make run	Start the API on http://localhost:7860
make test	Run the 125-test suite
make lint	Run ruff over src and tests
make docker-build / docker-run	Build and run the container
make report	Regenerate the diagrams and rebuild this document
docker compose up --build	FinAgent together with Qdrant

E. Example scripts

Script	Demonstrates
examples/01_evaluate_startup.py	The full pipeline: register, upload, verify, generate the PDF.
examples/02_investor_chat.py	Due-diligence Q&A grounded in one startup's materials.
examples/03_finscope_chat.py	Routing across all five subagents, plus guardrail probes.

Script	Demonstrates
examples/04_background_analysis.py	Fire-and-poll analysis for a UI that cannot block.
examples/curl_recipes.sh	Every endpoint as a copy-pasteable curl call.

F. Fixture data

Everything in `dataset/` is synthetic and written for this project — company names, founders, metrics and CINs are invented. No model here is trained; the directory exists for demonstration and for checks the offline unit suite cannot make.

File	Rows	Use
sample_startups.json	6	Profiles spanning strong, mixed and weak cases.
sample_cins.csv	10	Valid, malformed and struck-off CIN paths with expected status codes.
finscope_intents.jsonl	30	Labelled messages for checking the intent router.
guardrail_probes.jsonl	20	Prompts that must be blocked, and benign ones that must not.
sample_pitch_deck.md	1	A pitch deck in text form, for the ingestion path.